

Continuous-Control NEXUS: Evaluation of Hierarchical Skills, Symbolic Selection, Language-Generated Skills, and Visual Augmentation

Ivan Smirnov, Maryana Smirnova, Rosela Berberi, Anja Koroveshi
Technical University of Darmstadt

PREVIEW. Green highlight = added text; green frame = added figure.
Red = deleted text: none (no words were removed or changed).

Abstract—NEXUS represents a policy as a high-level choice among semantically named skills. We extend this structure from discrete actions to continuous control by replacing each low-level discrete skill policy with a bounded deterministic actor and state-action critic while retaining the discrete skill selector. We compare neural, symbolic, and neuro-symbolic (NeSy) variants with flat and hierarchical baselines across locomotion and manipulation tasks. The experiments test performance and robustness, the behavioral and causal role of individual skills, the contribution of symbolic selection, language-generated skill specifications, and visual augmentation. Neural and NeSy variants improve reliability over the matched flat learner on the Hopper and Go1 evaluations, but the advantage is task-dependent and does not imply superiority to PPO. Skill interventions show meaningful but partly redundant modules, symbolic rules work better as constraints than as a complete controller, and neither generated skills nor camera input provides a consistent performance benefit under the tested conditions.

Code: https://github.com/Tornadosky/nexus_project *Videos:*
<https://nexus-hrl.pages.dev/>

I. INTRODUCTION

Deep reinforcement learning can learn continuous control policies directly from state to torque, but the resulting mapping is difficult to inspect, modify, or constrain. Hierarchical reinforcement learning (HRL) introduces intermediate decisions, such as options or skills, that divide a control problem into recurring subproblems [2], [3]. NEXUS makes this intermediate structure explicit: lower-level policies are assigned semantic objectives, while a high-level meta-policy selects among them using a neural, symbolic, or neuro-symbolic rule [1]. The intended advantage is not only performance. A named skill can be forced, removed, or made conditionally available, which creates interventions that are hard to define for a monolithic policy.

The original NEXUS implementation targets discrete-action environments. In continuous control, each skill must instead produce bounded real-valued actions, while contact and momentum couple nominal modes such as moving, stabilizing, and recovering. Symbolic rules can therefore switch at inappropriate phases or cover the wrong region of state space. We test which parts of the semantic hierarchy remain useful under

these conditions. After introducing the variants, we refer to them simply as Neural, Symbolic, and NeSy.

The paper follows five research questions; each result subsection states the relevant experimental contrast and ends with a question-specific conclusion.

- Q1. Performance and robustness:** How do Neural and NeSy compare with a flat continuous learner, a hierarchy without skill-specific rewards, Symbolic, and PPO?
- Q2. Skill behavior:** Do the named continuous-control skills behave as intended, and how does removing an individual skill affect task success?
- Q3. Symbolic selection:** What does the symbolic component contribute when it is used as a fixed selector or as an eligibility mask around a learned selector?
- Q4. Language-generated skills:** Can LLM-generated skill specifications preserve task success, and does feedback refinement improve them?
- Q5. Visual augmentation:** Do camera observations improve state-aware skill policies over state-only and uninformative-image baselines?

Appendix A provides the language-generation protocol and supplementary language results.

II. CONTINUOUS-CONTROL NEXUS

A. Policy structure

Let s_t denote the state, $i \in \{1, \dots, K\}$ a skill, and a_t a bounded continuous action. Each skill has a deterministic actor and a state-action critic:

$$\pi_i(s) = b + c \odot \tanh f_{\theta_i}(s), \quad Q_i(s, a), \quad (1)$$

where b and c map the actor output to the action bounds. The meta-policy selects z_t , and only $\pi_{z_t}(s_t)$ is executed. The learned selector is a discrete value function $Q_\phi^M(s, i)$ over skills. We compare three variants:

$$\text{Neural: } z_t = \arg \max_i Q_\phi^M(s_t, i), \quad (2)$$

$$\text{Symbolic: } z_t = g(s_t), \quad (3)$$

$$\text{NeSy: } z_t = \arg \max_{i: m_i(s_t)=1} Q_\phi^M(s_t, i), \quad (4)$$

where g is a fixed priority rule and m_i indicates skill eligibility. Skill critics learn from their respective rewards, while the learned meta-Q uses the environment reward. Both use $Q(\lambda)$ targets [4]; each actor is updated to increase its own critic's value.

B. Changes from the original NEXUS implementation

We retain the NEXUS skill hierarchy [1] and make three implementation changes:

- 1) **Continuous low-level learner.** Following PureJaxQL's actor-critic PQN formulation, each skill uses a bounded actor $\pi_i(s)$ (Eq. 1) and a state-action critic $Q_i(s, a)$. In the $Q(\lambda)$ targets, we replace $\max_{a'} Q_i(s', a')$ with $Q_i(s', \pi_i(s'))$. All critics learn from shared transitions and executed actions using their own skill rewards; each actor maximizes its own critic.
- 2) **Exploration and skill selection.** We add Gaussian noise to the selected actor output and clip it to the action bounds. Neural and NeSy retain epsilon-greedy skill selection. In NeSy, exploration, greedy selection, and meta-Q bootstrapping respect the eligibility mask; excluded values are set to -10^9 rather than multiplied by zero. An empty mask falls back to all skills.
- 3) **Environment and task integration.** We add a batched MuJoCo Playground adapter for observations, action bounds, episode boundaries, and named physical state quantities. Task modules use these quantities for skill rewards, fixed selection rules, and NeSy masks, with measured and commanded velocities expressed in the same coordinate frame.

C. Symbolic content and an example rule

For CartpoleBalance, x is cart position relative to the centre in metres, θ is the pole angle from upright in radians, and $\dot{\theta}$ is its angular velocity in radians per second. The recovery flag $U(s)$ equals 1 when the pole tilts too far ($|\theta| > 0.20$) or rotates too fast ($|\dot{\theta}| > 1.5$), and 0 otherwise. The fixed selector chooses a skill:

$$g(s) = \begin{cases} \text{recover_balance}, & U(s) = 1, \\ \text{center_cart}, & U(s) = 0 \text{ and } |x| > 0.35, \\ \text{damp_motion}, & \text{otherwise.} \end{cases} \quad (5)$$

Thus, centering is selected only when recovery is not needed and the cart is more than 0.35 metres from the centre. NeSy instead uses separate rules to determine which skills are available, with damping always available. The learned meta-Q chooses among them, and the selected actor produces the continuous action.

III. EXPERIMENTAL SETUP

A. Environments, Rewards, Success Criteria, and Skills

The six environments were chosen to cover a rough progression in difficulty: Cartpole and Cheetah as simpler starting tasks, Walker and Hopper for locomotion with balance demands, and Panda and Go1 for manipulation and commanded

quadruped motion. Table I summarizes the tasks, success criteria, and hand-designed skills.

Exact reward terms, symbolic thresholds, and environment-specific configurations are provided with the released implementation.

B. Metrics

We distinguish environment return from task success. Episode return is

$$R_{\text{ep}} = \sum_{t=0}^{T-1} r_{\text{env},t}. \quad (6)$$

Because reward shaping and scale differ by environment, return is used to analyze learning within an environment rather than to compare tasks directly.

The primary success rate is a task-specific fraction of evaluation steps satisfying a binary predicate C_{task} :

$$\hat{S} = \frac{1}{T} \sum_{t=0}^{T-1} \mathbf{1}\{C_{\text{task}}(s_t) = 1\}. \quad (7)$$

Success therefore measures the fraction of evaluation steps satisfying the task condition, not the fraction of successfully completed episodes. For Go1, success requires safe body state together with bounded planar-velocity and yaw-rate tracking errors; for Hopper it requires upright forward hopping. Skill usage, forced execution, removal, selector interventions, and noise tests help explain policy behavior and task success.

C. Baselines and variants

Flat AC-PQN, labeled *Flat* in figures, removes the hierarchy and uses one continuous actor-critic. HPQN retains multiple actors and high-level selection but gives every skill the environment reward, isolating the effect of skill-specific objectives. PPO provides an external MuJoCo Playground reference [5], [6]. Neural versus NeSy isolates the symbolic mask, while Symbolic versus NeSy contrasts fixed rule selection with learned resolution among eligible skills.

D. Experimental protocol and data

State-based evaluations cover all six tasks. Language-generated skills are tested on Cartpole, Cheetah, Walker, Hopper, and Go1, including feedback refinement and repeated generation on Cheetah and Walker. Visual augmentation uses Cartpole, Cheetah, and Walker, with state-only, live-camera, and fixed-image conditions compared on Cartpole and Walker. We also evaluate training budget, action noise, Go1 command and terrain shifts, and skill and selector interventions. Run counts and uncertainty are reported with the figures and tables.

E. Why the experiments answer the research questions

- Q1. Flat tests the hierarchy against the matched implementation, HPQN isolates skill-specific rewards, PPO provides an external reference, and perturbation/shift tests probe reliability.
- Q2. Per-skill reward curves track the objectives during learning; forced rollouts test intended behavior, and removal

TABLE I
TASK DEFINITIONS, SUCCESS CRITERIA, AND HAND-DESIGNED SKILL SETS USED IN THE EXPERIMENTS.

Environment	Task objective	Success criterion	Hand-designed skills
CartpoleBalance	Balance the pole and limit cart motion.	Upright pole; cart inside the evaluation region.	<code>recover_balance</code> : restore uprightness; <code>center_cart</code> : move toward center; <code>damp_motion</code> : reduce residual cart/pole velocity.
CheetahRun	Run forward with stable posture.	Sufficient forward speed; stable torso pitch.	<code>accelerate_forward</code> : maximize forward motion; <code>stabilize_posture</code> : reduce pitch and excessive joint motion; <code>energy_efficient_run</code> : retain speed while penalizing large actions.
WalkerWalk	Walk forward while upright.	Upright torso; positive forward velocity.	<code>stand_recover</code> : restore height/posture; <code>walk_forward</code> : promote forward walking; <code>stabilize_gait</code> : suppress unstable motion; <code>energy_efficient</code> : preserve motion with smaller actions.
HopperHop	Hop forward while upright.	Standing condition; sufficient forward speed.	<code>stand_recover</code> : restore height/uprightness; <code>hop_forward</code> : promote forward hopping; <code>stabilize_landing</code> : reduce unstable landing dynamics; <code>energy_efficient</code> : retain standing/speed while penalizing large actions.
PandaPickCube	Reach, grasp, and lift the cube.	Successful cube lift.	<code>reach_cube</code> : reduce TCP-cube distance; <code>grasp_cube</code> : secure the object; <code>lift_cube</code> : increase cube height; <code>place_or_stabilize</code> : move or maintain the lifted cube.
Go1JoystickFlatTerrain	Track planar velocity and yaw commands.	Safe body state; bounded velocity/yaw errors.	<code>stand</code> : maintain stance; <code>track_velocity</code> : follow (c_x, c_y) ; <code>turn</code> : follow c_ψ ; <code>recover</code> : restore safe height/orientation.

tests how success changes when the selector must use the remaining skills.

- Q3. Holding trained actors fixed while removing the NeSy mask or replacing learned selection with a rule isolates the roles of filtering and skill choice.
- Q4. Language experiments keep the trainer fixed while changing the generated skill specification and its feedback-based revision.
- Q5. Visual experiments compare state-only, state plus live camera input, and state plus a fixed uninformative image while keeping the actor architecture comparable.

IV. RESULTS

A. Performance and Robustness

This subsection addresses Q1 using the state-based sweep, matched-budget Hopper and Go1 comparisons, learning curves, budget sensitivity, action-noise tests, and command/terrain shifts.

1) *Overview across environments*: Figure 1 reports final environment return relative to Flat. Neural improves Cheetah and Hopper and gives smaller gains on Cartpole and Walker, but falls below Flat on Go1. NeSy improves Cartpole and Cheetah but remains below Flat on the other tasks shown. Symbolic is generally weaker, while Panda is near or below Flat for all three variants. The results therefore depend on both the task and the selection method.

2) *Matched-budget comparison*: Figure 2 compares task success after training. On both focal tasks, every Neural seed and every NeSy seed finishes above every Flat seed. The interpretation differs across environments. On HopperHop, HPQN also learns a substantial part of the task, indicating that several behavioral modes are already useful even when their rewards are not specialized. On Go1JoystickFlatTerrain, HPQN collapses near the Flat failure regime, while Neural and

NeSy remain substantially higher. The Go1 result therefore depends on skill-specific objectives rather than merely on having several actor heads.

The PPO comparison prevents an over-general conclusion. PPO is weak on Hopper under this matched setup, while it achieves higher average success on Go1; the best Neural run nevertheless exceeds the PPO runs shown. Neural and NeSy can therefore improve the matched Flat implementation without being universally preferable to a strong task-specific optimizer.

3) *Learning dynamics*: Figure 3 shows training return across all six environments. On Hopper, Neural and NeSy separate from Flat as training progresses; on other tasks, the methods differ in learning speed, final return, and variability. PPO learns successfully on Cheetah, so its weak Hopper result is not a general failure of the baseline. These curves complement the task-success comparisons by showing how learning develops.

4) *Budget sensitivity*: The Hopper budget ladder tests whether longer training is enough for Flat to match the hierarchical policies (Fig. 4). Flat improves initially but gains little at the largest budgets. Neural reaches high success earlier along the ladder, while NeSy reaches a similar level with more training. Some Flat runs overlap the hierarchical results, but increasing Flat’s budget does not consistently close the performance difference.

5) *Action-noise robustness*: During evaluation, we add Gaussian noise to the chosen action to test whether policies remain successful when control becomes less precise. The middle and right panels of Fig. 5 show the Go1 and Hopper results. Neural and NeSy remain above Flat throughout the tested noise range, although their success generally decreases. Their similar responses do not establish an additional noise-robustness benefit from the symbolic mask. The left panel

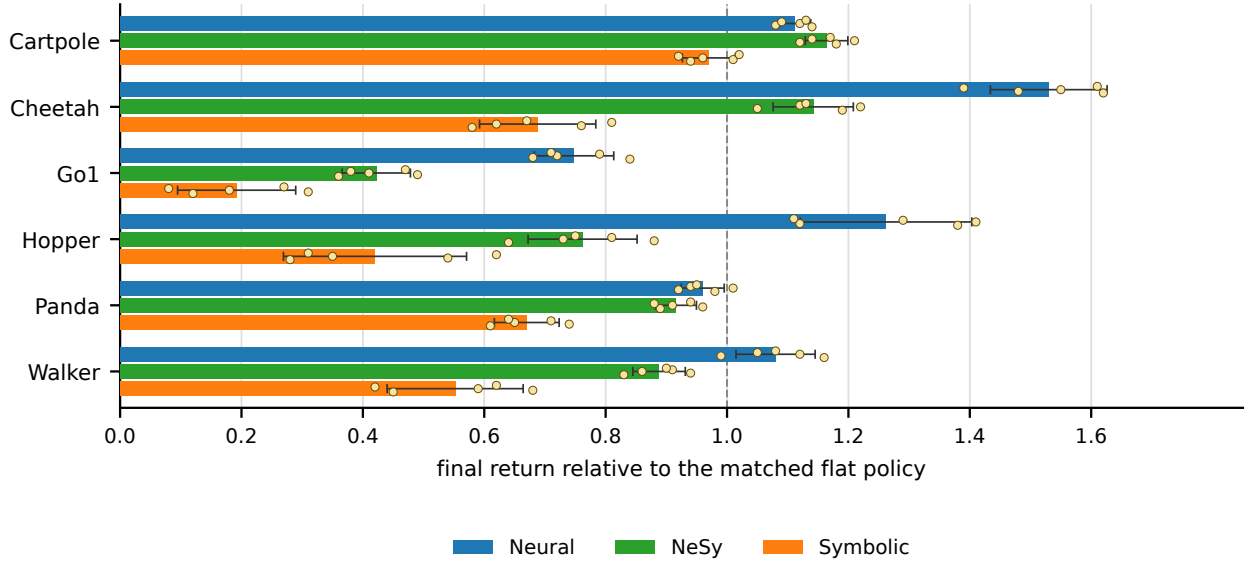


Fig. 1. State-based sweep, shown as final return relative to a matched Flat policy. Values above one favor the hierarchical variant. The mixed pattern motivates environment-specific analysis rather than a single global performance claim.

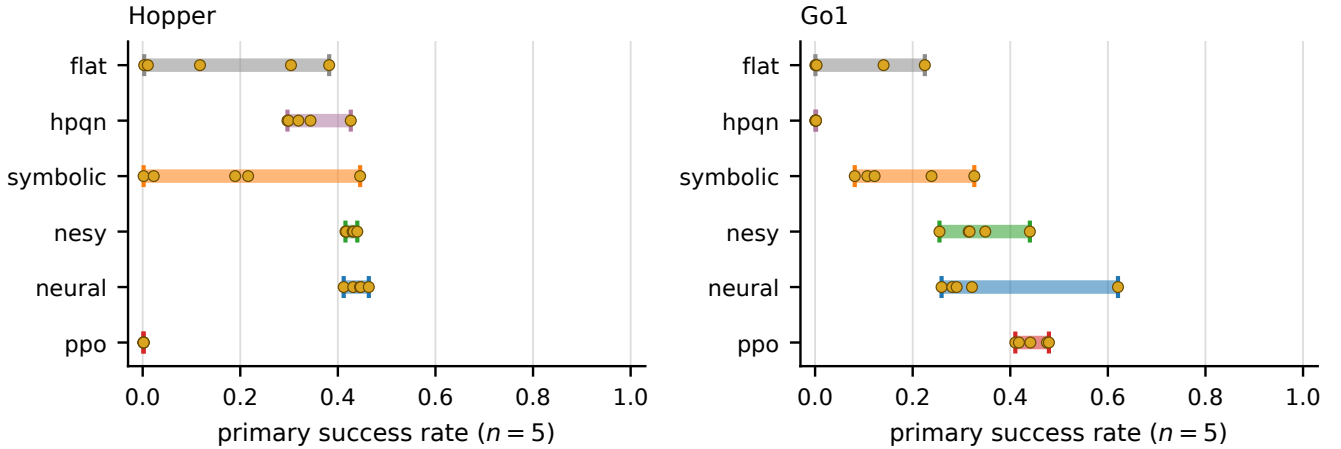


Fig. 2. Matched-budget primary success on Go1JoystickFlatTerrain and HopperHop. Points are seeds. Neural and NeSy finish above Flat on both tasks; HPQN behaves differently across tasks, and PPO reverses its relative position between Go1 and Hopper.

reports rough-terrain Go1 performance, where PPO performs best.

6) *Go1 command and terrain shifts*: Figure 6 evaluates rough-terrain Go1 policies under changed conditions without retraining. A command is the desired planar velocity (c_x, c_y) together with desired yaw rate c_ψ . “Stop” sets the motion command to zero, 1.5 and 2.0 m/s increase the forward command, and the condition labeled “flat” evaluates the same rough-trained controller on flat terrain.

The stop condition removes the motion command, so standing safely can satisfy the tracking objective. Higher forward commands reduce success, while rough-to-flat transfer retains some task performance. NeSy generally improves the lowest observed success relative to Flat, whereas Neural shows a wider spread. These results support a task-specific reliability

benefit, not a guarantee of successful tracking under every shift.

Q1 conclusion. Neural and NeSy outperform Flat in the Hopper and Go1 task-success comparisons and remain stronger under action noise. On Go1, the weak HPQN result supports the importance of skill-specific objectives. NeSy improves lower-end performance under the tested shifts, but PPO achieves higher average Go1 success and is much weaker on Hopper. Hierarchy therefore helps under the tested conditions without providing a universal performance advantage.

B. Skill Behavior

This subsection addresses Q2 using skill-reward curves, forced execution, and skill removal.

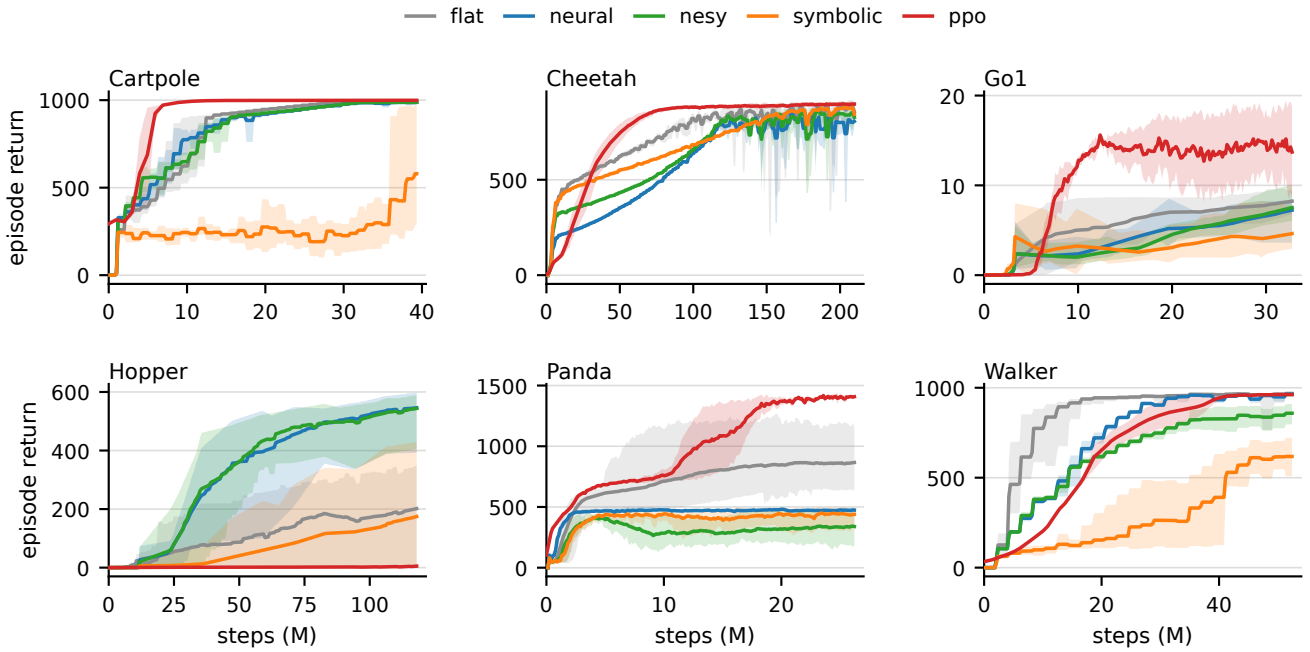


Fig. 3. Training return versus environment interaction steps for all six state-based environments. The curves show learning speed, later separation, and variability that post-training summaries omit. Training durations are shown on the horizontal axes; return and task success measure different aspects of performance.

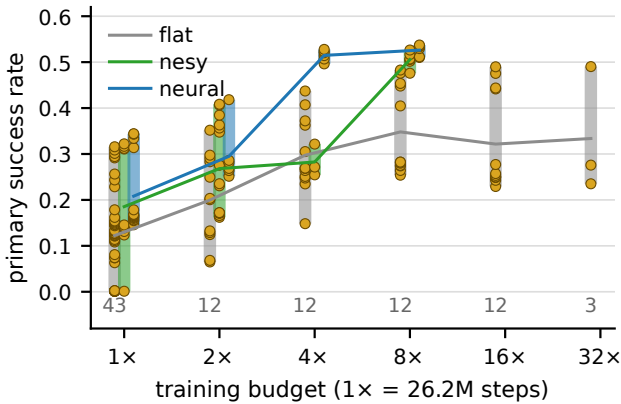


Fig. 4. Hopper task success across increasing training budgets. Flat improves initially but gains little at the largest budgets. Neural reaches high success earlier along the ladder, and NeSy reaches a similar level with more training; some individual Flat runs overlap these results.

1) *Skill-reward learning*: Figure 7 tracks each hand-written skill reward during NeSy training across all six tasks. The forward-motion rewards grow on Cheetah and Walker, while Cartpole’s recovery reward has a large negative early transient. Each curve sums that reward over the whole episode, including steps when another skill acts. It therefore tracks the corresponding objective, not how much that actor contributes to task return. Reward magnitudes also differ by design, so behavioral and removal tests are needed to establish what the skills do. Appendix B repeats the measurement on Go1 and Hopper for the Flat, HPQN, and PPO baselines (Fig. 17) and for the Neural and Symbolic variants beside NeSy (Fig. 18);

the other four tasks have no matched baseline logs.

2) *Behavioral meaning*: We freeze trained NeSy policies and force each skill to act for a diagnostic rollout. Seven of eight tested Go1 and Hopper skills move the state quantity associated with their semantic objective in the intended direction (probe labels in Fig. 8). The main exception is Hopper’s *energy_efficient* skill, whose forced behavior is not cleanly distinguished from the other hopping modes. Forced execution therefore tests semantic behavior independently of how often the selector normally uses a skill.

3) *Skill removal*: We next remove one actor from a trained NeSy policy and let the selector reroute through the remaining skills. Most removals preserve substantial success (Fig. 8), including removal of Go1 *recover* in many runs. The skill library is therefore behaviorally legible but partly redundant: forced execution tests the intended behavior, while removal tests how much the trained policy depends on that actor.

Q2 conclusion. The tested NeSy actors are mostly behaviorally consistent with their semantic names, but the skill library is partly redundant: removing one actor often preserves substantial task success because the learned selector reroutes through other skills. The hierarchy is therefore best described as an interpretable set of behavioral modes rather than a collection of strictly independent causal modules.

C. Symbolic Selection

This subsection addresses Q3.

1) *Selector interventions with fixed actors*: We keep trained NeSy actors fixed and change only how skills are chosen. In Fig. 9, *as trained* retains NeSy, *mask removed* allows the learned selector to choose any skill, and *symbolic only* replaces it with the hand-written rule.

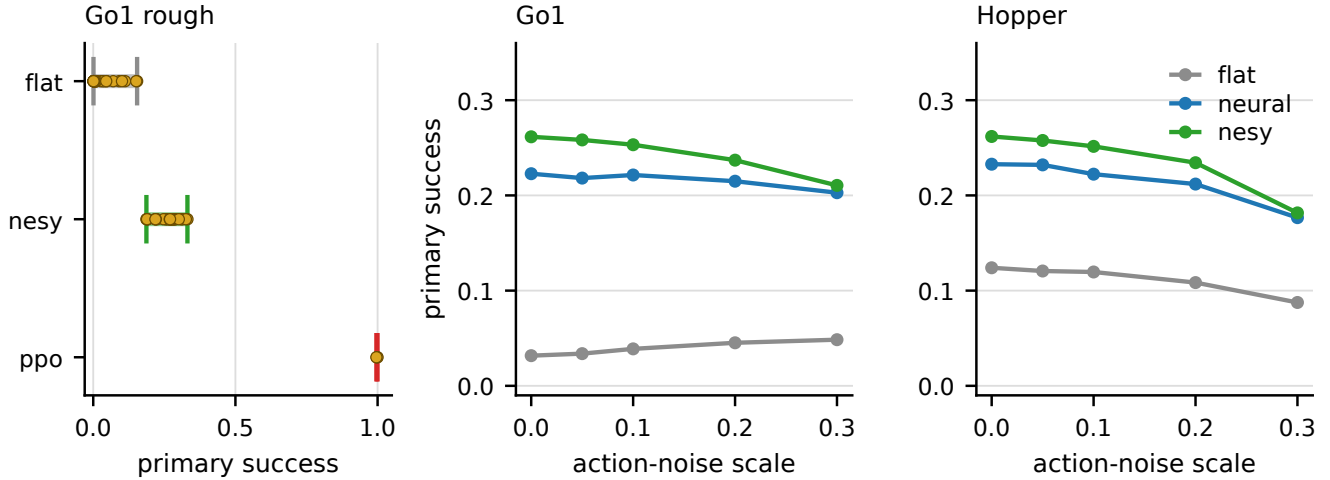


Fig. 5. Reliability comparisons. Left: primary success for rough-terrain Go1 evaluation. Middle and right: primary success under increasing evaluation-time action noise on Go1 and Hopper. Neural and NeSy remain above Flat over the tested noise range. Absolute success is shown rather than change normalized to each method’s baseline.

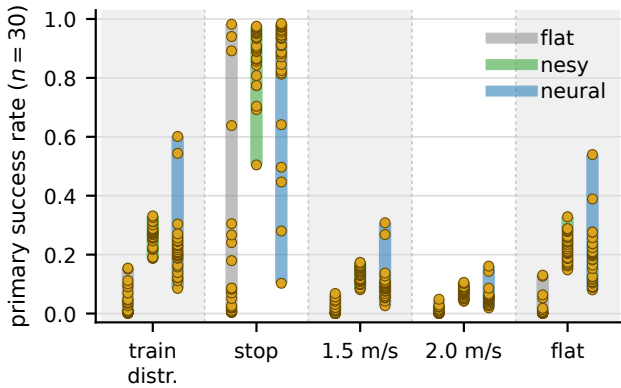


Fig. 6. Go1 command and terrain shifts for rough-trained policies. “train distr.” is evaluation on the training distribution; “stop” sets the command to zero; 1.5 and 2.0 m/s increase the forward command; “flat” denotes rough-to-flat transfer without retraining.

On Hopper, removing the mask changes little, whereas replacing learned selection with the fixed rule reduces success. On Go1, removing the mask lowers the weakest results, while fixed selection does not match the best NeSy result. These interventions distinguish restricting the available skills from deciding which available skill should act.

2) *Fixed symbolic selection*: The Symbolic baseline uses hand-written rules to choose the active skill during both training and evaluation. In Fig. 2, it is less consistent on Hopper and generally weaker on Go1 than Neural and NeSy. Together with Fig. 9, this favors learning which skill to use rather than assigning every choice to a fixed rule.

Q3 conclusion. Learning which skill to use is more reliable than relying entirely on the fixed rule. The mask changes little on Hopper but improves the weakest Go1 outcomes. Here, symbolic rules are more useful for restricting choices than for making every decision.

D. Language-Generated Skill Specifications

This subsection addresses Q4.

1) *Generation and feedback refinement*: The language extension keeps the reinforcement-learning trainer fixed and replaces the hand-written skill specification. The generator receives a task description, the available semantic state fields, a requested skill count, and a bounded vocabulary of executable reward terms and activation rules. Its output is parsed into skill names, activation conditions, and weighted reward terms; arbitrary generated code is never executed. A validator checks the fields, operators, reward terms, skill count, and always-valid fallback before training.

Feedback refinement returns evaluation statistics to the language model and trains the revised specification from scratch. We compare initial proposals, feedback revisions, and independently resampled proposals; Table III also records reward and skill count across refinement iterations. For example, one Cheetah proposal selected `sprint` on 98.6% of steps. After feedback, the model increased its forward-velocity weight from 5 to 8, changed the `stabilize` trigger from `|torso pitch| > 0.5` to `> 0.2`, and lowered the `smooth_motion` trigger from `joint speed > 10` to `> 6`. These revisions change the skill specification rather than continue training the same policy.

2) *Results across tasks*: Figure 10 compares hand-written, initial generated, and feedback-refined specifications across five environments. Cheetah remains close to the hand-written reference in both reward and success. Cartpole and Walker initially lose substantial success despite nonzero shaped reward; refinement improves their results but does not restore the reference performance. Hopper remains near failure, and Go1 success stays low. Table II gives the numerical comparison, while Table III records the reward and skill-count changes during refinement.

Figure 16 in Appendix A shows skill usage on Cartpole and Cheetah. Cartpole’s initial generated policy selects one skill

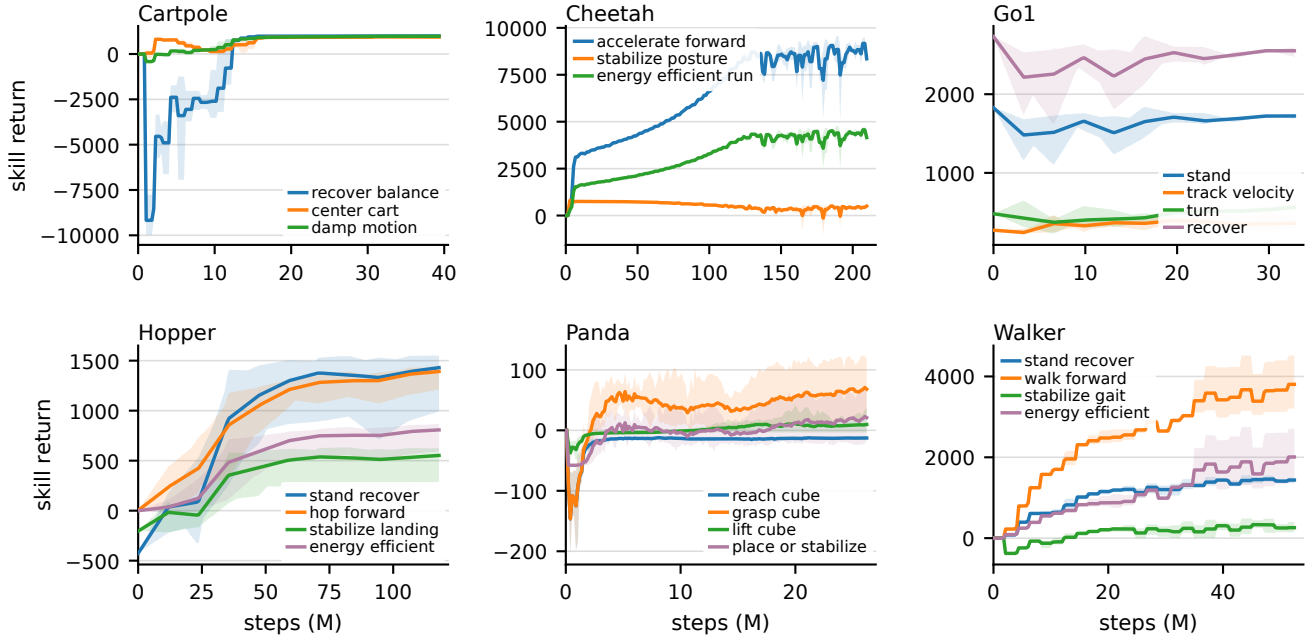


Fig. 7. Per-skill reward accumulated over the episode versus environment interaction steps for hand-written NeSy policies. Each line corresponds to one skill reward and each band is the seed min-max. Rewards include all steps, even when another skill acts, so they do not measure the selected actor’s contribution to environment return. Sample sizes are $n = 3$ for Cartpole, Cheetah, and Walker, $n = 9$ for Panda, and $n = 5$ for Go1 and Hopper. Reward scales differ between skills and tasks.

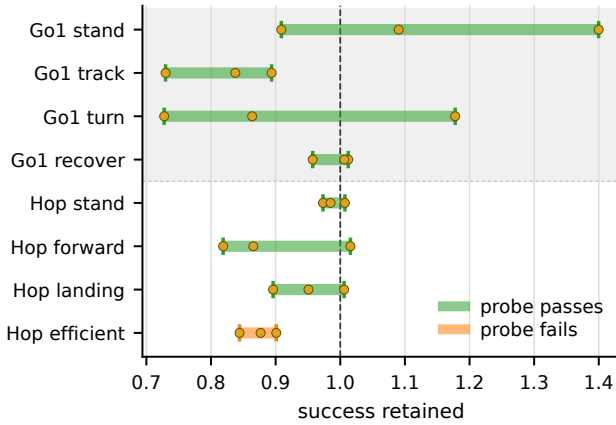


Fig. 8. Single-skill removal from trained NeSy policies, shown as success relative to each native policy. Values near one indicate retained performance; values above one indicate improvement after removal. Colors mark whether the corresponding forced-skill behavior probe passes or fails. Many removals retain substantial success.

throughout the rollout; refinement spreads selection across several skills but does not recover hand-written performance. Usage describes which skills are selected, whereas Fig. 10 measures how well the resulting policy performs.

Figure 11 shows how task success varies across generated specifications on Cheetah and Walker. Cheetah generally stays near the hand-written reference, although resampling includes weaker runs. Walker’s initial proposals range from low success to reference-level performance; feedback revision does not consistently resolve this variation, and the resampled proposals perform poorly. Resampling tests dependence on the particular generated decomposition, while refinement tests whether



Fig. 9. Selector interventions with trained NeSy actors held fixed. “as trained” retains the original selector; “mask removed” disables symbolic filtering; “symbolic only” replaces learned selection with the fixed rule. Removing the mask has little effect on Hopper but lowers the weakest Go1 results.

feedback improves the original proposal. Neither is simply additional training of the same policy.

Schema validation occurs before reinforcement learning, so invalid specifications never enter training. One language model configuration failed the bounded schema on every planned generation attempt, while another produced executable specifications under the same interface. Appendix A gives the generation details.

Q4 conclusion. Language-generated specifications can approach the hand-written reference, most consistently on Cheetah, but executable output does not guarantee a useful decomposition. Walker exposes strong dependence on the generated proposal, and feedback refinement does not reliably recover hand-written performance.

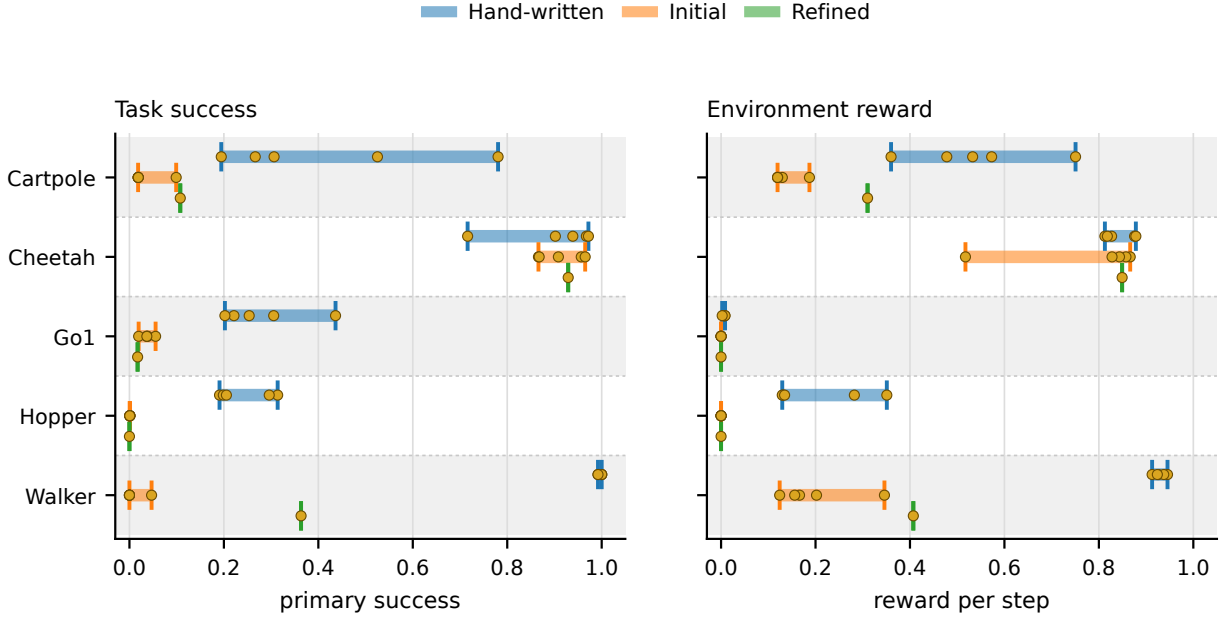


Fig. 10. Language comparison across five environments. Left: deterministic primary success. Right: environment reward per step. Cheetah remains close to the hand-written reference. Initial generated Walker skills receive nonzero reward but almost no task success; the refined run improves success while remaining below the reference. Hand-written and initial conditions show repeated runs; refined is a single final feedback run per environment.

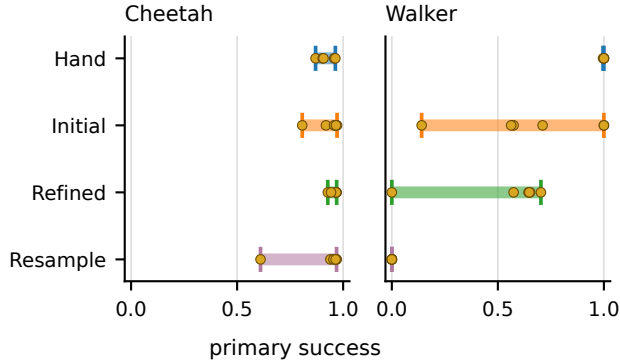


Fig. 11. Task success on Cheetah and Walker at a matched RL training budget. Hand-written NeSy is compared with initial, once-refined, and independently resampled specifications. All are trained from scratch. Cheetah is comparatively stable; Walker varies across proposals, and resampling performs poorly.

E. Visual Augmentation

This subsection addresses Q5.

1) *Scope and visual input:* The comparisons reported here test camera augmentation, not vision-only control. Every condition retains the full privileged state; when enabled, the camera pathway augments only the low-level skill actors. Despite the “RGB” label used in code and some figures, the actor input is a 64×64 grayscale image stack containing three consecutive frames; the stack supplies temporal information rather than color channels. The image encoder produces features that are concatenated with the same state vector used by the state-only actor. Skill critics, meta-Q, symbolic mask, skill-reward feature extraction, and task-success predicates remain state-based.

The state-only condition uses the same vision-configured

environment but disables the actor’s camera pathway, keeping the task, reward, and state representation unchanged. The fixed-image condition, labeled *Constant* in figures, keeps the encoder and its extra parameters but feeds the same frozen image at every time step. It therefore tests whether any change comes from informative visual content rather than simply adding a CNN pathway. Representative camera inputs and supplementary rollouts are available on the project website linked at the beginning of the paper.

2) *Performance and visual information:* Figure 12 reports seed-matched changes in evaluation score relative to state-only on Cartpole, Walker, and Cheetah. Walker often improves, but not in every run; Cartpole includes substantial losses, while Cheetah has smaller mixed changes. The pattern does not support a general benefit from adding the camera pathway.

Figure 13 compares state-only, state+camera, and fixed-image performance on Cartpole and Walker. The overlapping ranges show no consistent camera advantage, although overlap alone does not establish equivalence. The fixed-image condition can perform comparably to live camera input, so the results do not clearly attribute any benefit to informative visual content rather than the added encoder pathway.

Q5 conclusion. Adding the camera pathway does not provide a consistent performance advantage over state-only input. Comparable fixed-image performance also leaves no clear evidence that live visual information improves control under the tested conditions.

V. DISCUSSION

A. What the hierarchy contributes

The results suggest that hierarchy can help organize learning rather than guarantee higher performance. Neural and NeSy

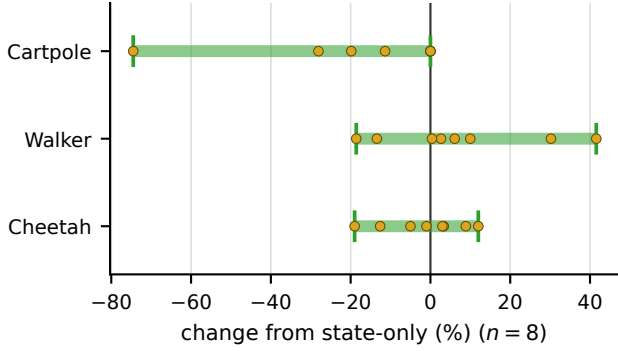


Fig. 12. Seed-matched percentage change in the reported evaluation score from state-only to state+camera on Cartpole, Walker, and Cheetah. Positive values indicate improvement; negative values indicate a decrease. These relative changes should not be interpreted as a common task-success scale across environments.

improve on Flat AC-PQN, but the HPQN comparison shows that the source of the improvement is task-dependent: multiple actors already help on Hopper, while skill-specific objectives matter on Go1.

B. Behavioral interpretability versus modular necessity

Here interpretability means that named skills correspond to observable behavioral modes and permit direct interventions. Forced execution tests behavioral meaning, removal tests causal contribution, and selector editing tests symbolic structure. The experiments show meaningful but overlapping skills, so the hierarchy exposes an interpretable control interface rather than a set of strictly independent modules.

C. Reliability and competitiveness are different questions

A matched Flat ablation tests the effect of the hierarchy, while PPO provides an external performance reference. Neural and NeSy outperform Flat on Hopper and Go1 task success, but PPO achieves higher average success on Go1 and lower success on Hopper. Improving Flat therefore does not establish universal superiority over other RL methods.

D. Language-generated structure

Language generation is useful only after two checks: the specification must be executable, and independently generated skill sets must produce consistent task behavior. The experiments show that schema validity is necessary but not sufficient, and aggregate training feedback may miss structural errors in reward terms or activation conditions.

E. Visual augmentation

The camera experiments show why an input pathway needs an information baseline. A policy may become sensitive to images without gaining task performance, particularly when the privileged state already contains the relevant physical variables. The fixed-image condition makes that distinction visible without removing the extra encoder capacity.

F. Limitations

The study uses manually defined task-success predicates and hand-designed semantic state features. Those choices make the interventions clear but also constrain what the hierarchy can represent. The task set is small relative to the diversity of continuous-control problems, and the strongest performance claims are concentrated on Hopper and Go1. The language pipeline uses a bounded reward/rule vocabulary rather than unrestricted program synthesis. The visual study keeps the high-level selector and critics state-based, so it does not test a fully visual hierarchy. The conclusions apply to the reported training budgets and evaluation settings.

VI. REPRODUCIBILITY AND CONTRIBUTIONS

Code, configurations, analysis utilities, and experiment artifacts are available in the project repository. The results archive records experiment provenance, seed identities, and evaluation products used for the figures and appendix tables.

Rosela Berberi developed and evaluated the visual-input extension and experiments. Anja Korovesi developed and evaluated the language-generated skill pipeline, including skill generation, feedback refinement and comparison of skill usage across environments. Ivan Smirnov and Maryana Smirnova jointly developed the continuous-control framework and ran the main experiment suite. Ivan focused on algorithm integration, selector and baseline configurations, and experiment orchestration; Maryana focused on environment and reward integration, evaluation tooling, and robustness and skill-intervention analyses. Both contributed to experimental design and result analysis, and all authors contributed to reviewing the final manuscript. Outside the skill-generation experiments, we used LLMs for code development, paper revision and writing. The authors checked this work through personal verification.

VII. CONCLUSION

We adapted NEXUS to continuous actions by replacing each discrete low-level policy with a deterministic actor and state-action critic while preserving the discrete semantic selector. Neural and NeSy substantially improve task success and reliability over Flat AC-PQN on Hopper and Go1, but their relation to PPO is task-dependent. The learned skills are mostly behaviorally meaningful but partly redundant, and symbolic rules are most useful as constraints around learned selection rather than as a complete timing policy. Language-generated skills can approach hand-written performance, particularly on Cheetah, but results vary across generated specifications and feedback refinement does not reliably repair weak proposals. Adding a 64×64 grayscale three-frame camera pathway to state-aware skill actors does not yield a consistent performance advantage. The main contribution is therefore an experimentally testable account of when semantic structure changes continuous-control learning, which parts of that structure matter, and how generated skills and visual inputs can fail.

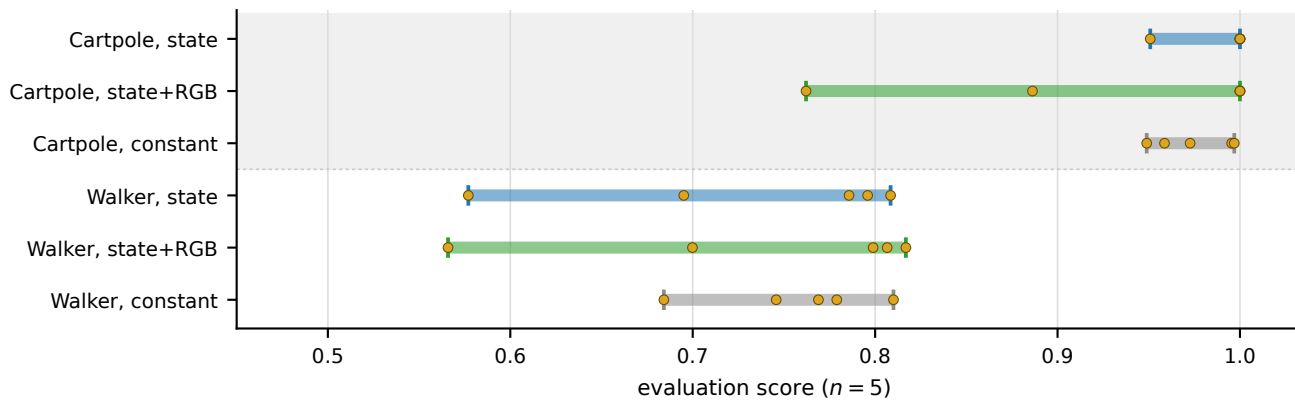


Fig. 13. Visual-input evaluation scores on Cartpole and Walker for state-only, state+camera, and fixed-image NeSy. “state+RGB” denotes the camera pathway and “constant” the fixed-image condition. The overlapping ranges show no clear, consistent camera advantage in this comparison.

REFERENCES

- [1] Anonymous Authors, “From Objects to Skills: Interpretable Meta-Policies for Neural Control,” under review, 2026.
- [2] R. S. Sutton, D. Precup, and S. Singh, “Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning,” *Artificial Intelligence*, vol. 112, no. 1–2, pp. 181–211, 1999.
- [3] P.-L. Bacon, J. Harb, and D. Precup, “The Option-Critic Architecture,” in *Proc. AAAI Conf. Artificial Intelligence*, 2017.
- [4] M. Gallici et al., “Simplifying Deep Temporal Difference Learning,” arXiv:2407.04811, 2024.
- [5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.
- [6] K. Zakka et al., “MuJoCo Playground,” arXiv:2502.08844, 2025.

TABLE II

LANGUAGE-GENERATED SKILL RESULTS BY ENVIRONMENT. HAND-WRITTEN AND INITIAL GENERATED VALUES ARE MEAN $\pm$ STANDARD DEVIATION OVER FIVE SEEDS; REFINED VALUES ARE FROM ONE FINAL FEEDBACK RUN PER ENVIRONMENT. REWARDS ARE ENVIRONMENT REWARD PER STEP, WITH ENVIRONMENT-SPECIFIC SCALES.

Environment	Hand-written	Initial	Refined
Environment reward			
Cartpole	0.539 ± 0.128	0.135 ± 0.026	0.310
Cheetah	0.842 ± 0.029	0.783 ± 0.133	0.849
Walker	0.929 ± 0.011	0.199 ± 0.078	0.407
Hopper	0.206 ± 0.093	0.000 ± 0.000	0.000
Go1	0.0064 ± 0.0021	≈ 0	≈ 0
Primary success rate			
Cartpole	0.415 ± 0.214	0.035 ± 0.032	0.108
Cheetah	0.899 ± 0.095	0.913 ± 0.042	0.929
Walker	0.997 ± 0.004	0.009 ± 0.019	0.363
Hopper	0.241 ± 0.053	0.001 ± 0.000	0.000
Go1	0.284 ± 0.084	0.037 ± 0.011	0.017

TABLE III

SEPARATE EXPLORATORY FEEDBACK-REFINEMENT TRAJECTORIES FROM ITERATION 0 TO ITERATION 3, ALLOWING THE SKILL COUNT TO CHANGE.

EACH ITERATION IS ONE TRAINING RUN. THESE ARE NOT THE FIXED-COUNT, ONE-REVISION COMPARISONS IN FIG. 11, AND ITERATION 0 IS NOT THE MULTI-SEED INITIAL MEAN IN TABLE II.

Environment	Reward $0 \rightarrow 3$	Skills	Outcome
Cartpole	$0.140 \rightarrow 0.310$	$3 \rightarrow 5$	peak at iter. 1
Cheetah	$0.743 \rightarrow 0.849$	$3 \rightarrow 5$	monotonic increase
Walker	$0.637 \rightarrow 0.407$	$3 \rightarrow 5$	net regression
Hopper	$0.000 \rightarrow 0.000$	$3 \rightarrow 5$	no reward signal
Go1	$0.0062 \rightarrow 0.0000059$	$3 \rightarrow 5$	collapse after iter. 0

APPENDIX A

LANGUAGE-GENERATED SKILL PROTOCOL

The language pipeline generates structured skill definitions before RL training. Prompts specify the task, available semantic fields, executable reward terms, activation-rule syntax, and requested skill count. Figure 14 shows the Cheetah and Walker examples. The generator configuration reported here is Qwen/Qwen3.8-27B; Qwen2.5-1.5B-Instruct did not produce executable specifications under the schema checks.

A generated proposal is trained without manually modifying its reward terms or activation rules. Evaluation statistics are then returned to the generator for feedback refinement. Figure 15 shows a Cheetah example that requests one revision while keeping the skill count fixed. The generation context and existing proposal are retained during this revision.

The revised specification is trained from scratch under the same RL configuration; only the generated skill specification changes.

APPENDIX B

SKILL RETURNS FOR BASELINES AND VARIANTS

Figures 17 and 18 score the hand-written Go1 and Hopper skill rewards on every method’s evaluation behavior, using the same quantity as Fig. 7. Flat, HPQN, and PPO were not trained on these rewards, so their curves show how much of

each skill objective the policy’s behavior satisfies, not skill learning. On Hopper, Neural and NeSy reach the highest return on all four skills, HPQN follows on `hop_forward`, and PPO stays near or below zero. On Go1 the Flat, HPQN, and hierarchical curves end close together, whereas PPO is higher on `stand`, `turn`, and `recover`. Bands are seed min–max with $n = 5$ and overlap substantially on Go1, so these are descriptive comparisons.

Skill-generation prompt

Design a continuous-control NEXUS skillset. Return JSON only:

```
{ "skills": [ { "name": "...", "activation_rule": "...",  
  "reward_terms": [ { "type": "...", "weight": 1.0, "lhs": "..."} ] } ] }.
```

Every scalar field is the same semantic feature supplied to the hand-designed controller. Fields and units: `forward_velocity` in m/s, `torso_pitch` in radians, `joint_speed` mean absolute joint angular speed in rad/s, and `height` in metres only when listed. No hidden fields or environmental reward are available.

Use EXACTLY the requested skill count and no extra term keys. One skill must have `activation_rule` "True". Others may overlap. NeSy chooses max learned meta-Q among allowed skills every step. Define simple complementary goals, not named copies.

Rules: numeric comparisons, and/or/not, + - *, `abs(x)`, `min(x,y)`, `max(x,y)`. No division.

Rewards sum terms. All weights > 0 and ≤ 10; penalty types negate their weight internally.

Allowed vocabulary and exact executable semantics:

`negative_distance`: `-weight*abs(lhs-rhs)`, `-weight*abs(lhs-threshold)`, or `-weight*abs(lhs)`.

`positive_velocity`: `weight*lhs`.

`target_height`: `weight*clip(lhs/threshold,0,1)`; requires positive threshold, not rhs.

`binary_bonus`: `weight*(lhs>threshold)`; requires lhs and threshold.

`action_penalty`: `-weight*sum(action**2)`; has neither lhs, rhs, nor threshold.

`posture_penalty`: `-weight*abs(lhs)`.

rhs must name another available field, never a number. Constants use `threshold` when supported. A terminal penalty of 1 is subtracted from every skill. Produce between one and six terms per skill.

Cheetah task prompt

Task: cheetah. Run forward quickly while maintaining posture.

Available fields: `forward_velocity`, `torso_pitch`, and `joint_speed`.

Skill count: 3.

Walker task prompt

Task: walker. Walk forward around 1 m/s while remaining upright near 1.2 m torso height.

Available fields: `height`, `torso_pitch`, `forward_velocity`, and `joint_speed`.

Skill count: 4.

Fig. 14. Language-model prompts used to generate continuous-control skill specifications. The common generation prompt constrains the output to executable reward terms and activation rules; the environment-specific message supplies the task, available semantic fields, and required number of skills.

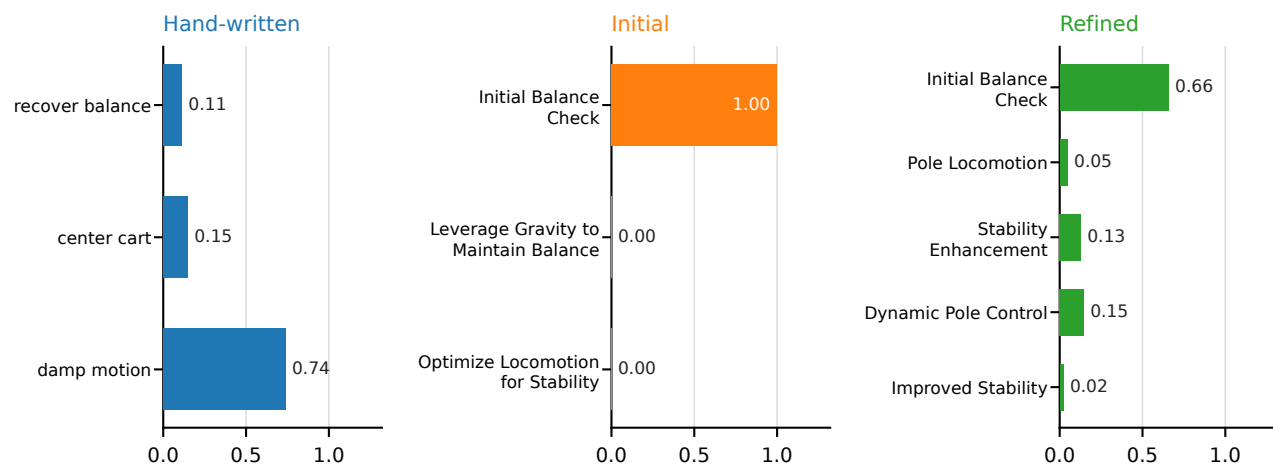
Cheetah feedback-refinement prompt

Revise this SAME proposal once using these pilot validation metrics. Do not add skills. All fields and reward constraints stay the same.

```
{  
  "episode_return_mean": 463.810302734375,  
  "episode_return_std": 59.37163543701172,  
  "episode_length_mean": 1000.0,  
  "action_square_mean": 0.5742929577827454,  
  "cheetah/action_norm": 1.8131752014160156,  
  "cheetah/forward_velocity": 4.647381782531738,  
  "cheetah/forward_velocity_mean": 4.647381782531738,  
  "cheetah/joint_speed": 5.230077743530273,  
  "cheetah/posture_stable_rate": 0.9785780906677246,  
  "cheetah/speed_success_rate": 0.9164843559265137,  
  "cheetah/torso_pitch": -0.05432555079460144,  
  "common_skill_step/0_accelerate_forward": 4.642935752868652,  
  "common_skill_step/1_stabilize_posture": 0.572869062423706,  
  "common_skill_step/2_energy_efficient_run": 2.305461883544922,  
  "primary_goal_metric": 4.647381782531738,  
  "primary_success_rate": 0.912656307220459,  
  "rules/eligible_count": 1.035656213760376,  
  "rules/overlap_fraction": 0.03551562875509262,  
  "skill_usage/0_sprint": 0.9855625629425049,  
  "skill_usage/1_stabilize": 0.013890624977648258,  
  "skill_usage/2_smooth_motion": 0.0005468750023283064,  
  "common_skill_return/0_accelerate_forward": 4642.935546875,  
  "common_skill_return/1_stabilize_posture": 572.8690795898438,  
  "common_skill_return/2_energy_efficient_run": 2305.4619140625  
}
```

Fig. 15. Example Cheetah feedback prompt. Here, the generator revises an existing proposal once while the task fields, skill count, reward vocabulary, RL trainer, and evaluation procedure remain fixed.

Cartpole



Cheetah

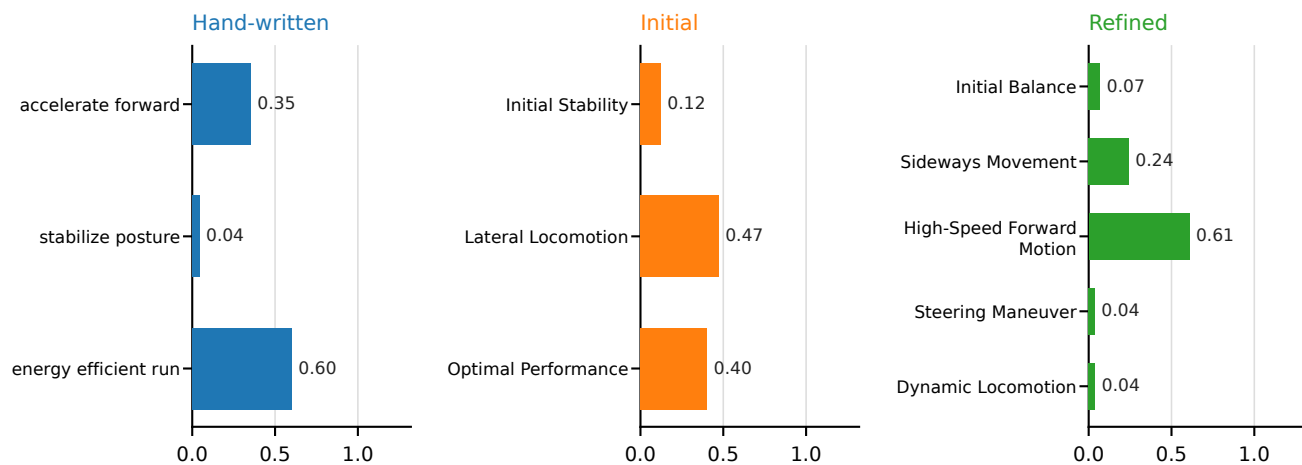


Fig. 16. Skill usage for Cartpole and Cheetah. Each block is the fraction of rollout steps spent in that skill. Generated sets can concentrate selection in a single generic skill; Cartpole's initial set uses one skill on every step.

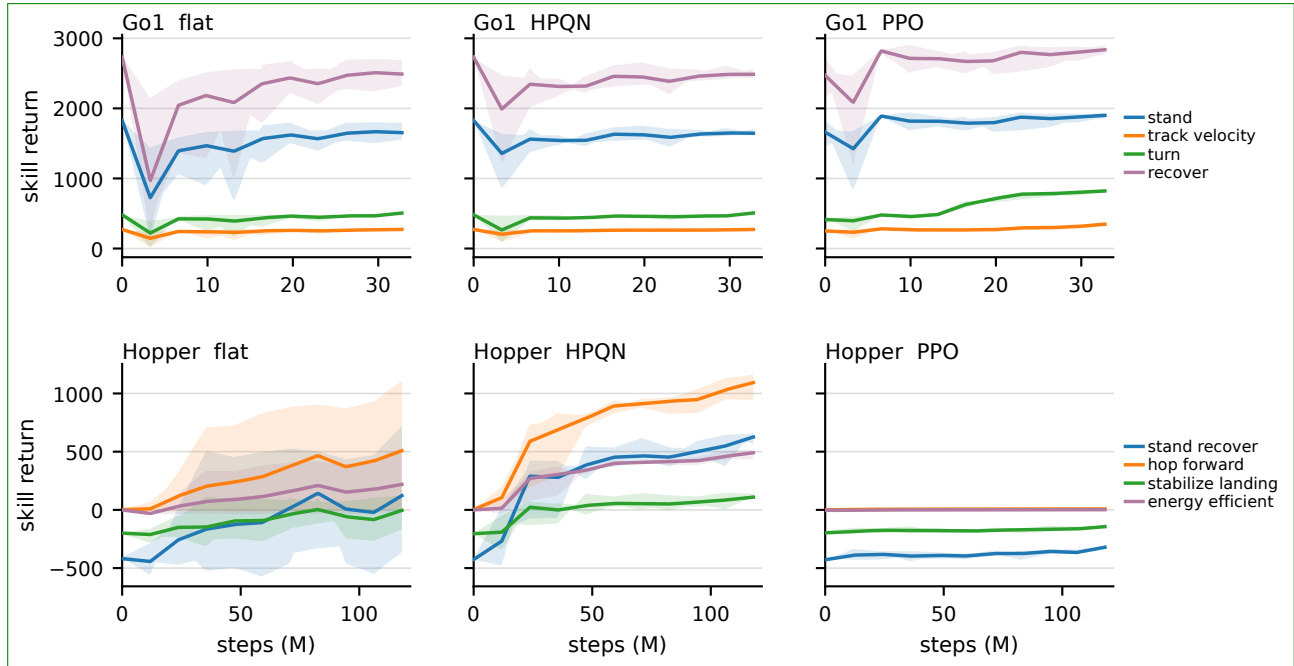


Fig. 17. Per-skill reward accumulated over the episode versus environment interaction steps for the Flat, HPQN, and PPO baselines on Go1 and Hopper, scored on the same hand-written skill rewards as Fig. 7. The baselines were not trained on these rewards. Each line is one skill and each band is the seed min-max ($n = 5$). Rewards include all steps, and reward scales differ between skills and tasks.

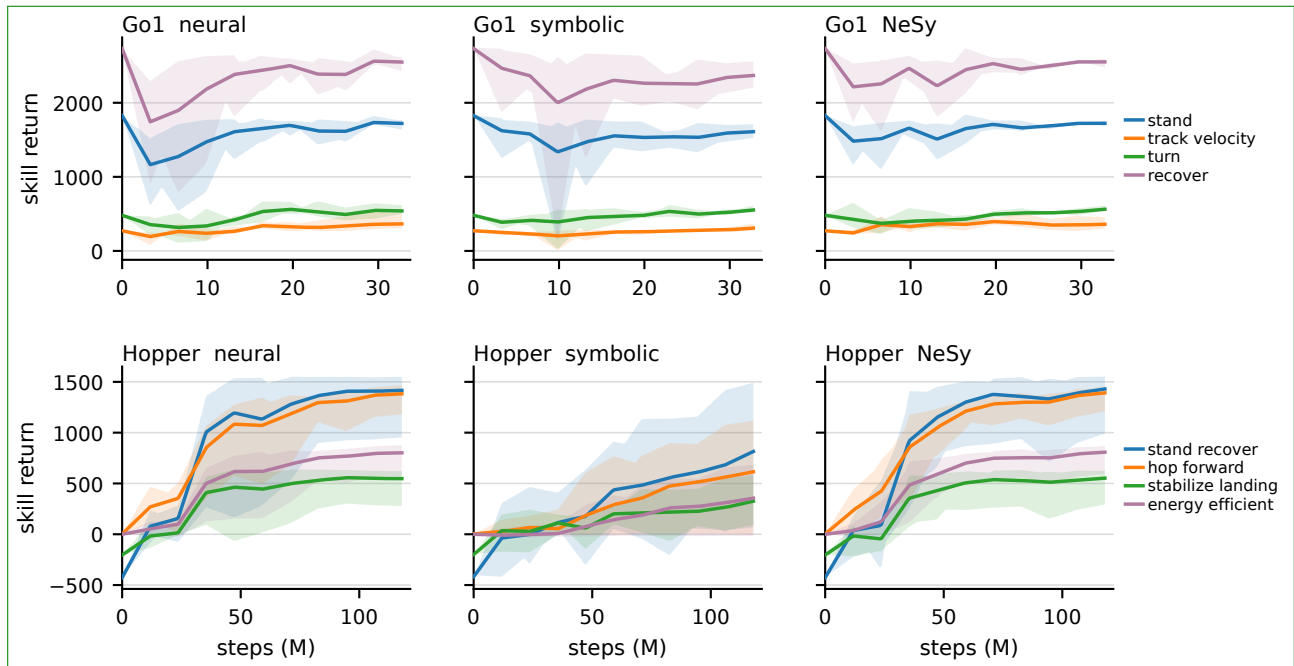


Fig. 18. Per-skill reward accumulated over the episode versus environment interaction steps for the Neural, Symbolic, and NeSy variants on Go1 and Hopper. The NeSy panels repeat the Go1 and Hopper panels of Fig. 7. Each line is one skill and each band is the seed min-max ($n = 5$). Rewards include all steps, even when another skill acts.